



UNIVERSITÉ DE MONTPELLIER

TRAVAIL ENCADRÉ DE RECHERCHE

Explorez l'IA créative : classez, colorisez et inventez des images uniques

Massila LAKAF, Imane LAZIZI, Kanzy YOUSSEF

*Encadrant : M.Pascal Poncelet
2025-2026*

Sommaire

1	Introduction	1
2	Rappels sur l'apprentissage automatique et l'apprentissage profond	1
2.1	Définition d'apprentissage profond et automatique	1
2.2	Concept Central d'Apprentissage Profond	2
2.2.1	Réseaux de Neurones	2
2.2.2	Fonction d'activation	3
2.2.3	loss function	3
2.2.4	descente de gradient	4
2.2.5	Backward Propagation	4
2.2.6	Les Réseaux de Neurones Convolutifs	5
3	Bibliographie	5

1 Introduction

CAHIER DES CHARGES :

Afin de mener à bien ce projet dans les délais impartis et de répondre aux objectifs fixés, nous avons établi le cahier des charges suivant. Le travail sera réalisé par un groupe de 3 étudiantes et se décomposera en plusieurs livrables

Objectifs fonctionnels :

- **Classification :** Développer un modèle de classification d'images convolutif performant sur un jeu de données donné. Le modèle devra être capable de prédire la catégorie d'une image.
- **Analyse et amélioration :**
- **Restauration d'images :**
- **Génération d'images :**
- **Application de démonstration :** Développer une interface web simple (frontend) permettant d'interagir avec les modèles entraînés.

2 Rappels sur l'apprentissage automatique et l'apprentissage profond

2.1 Définition d'apprentissage profond et automatique

l'apprentissage automatique désigne une famille de méthodes où un ordinateur améliore ses performances sur une tâche en apprenant à partir d'exemples plutôt qu'en suivant des règles explicitement programmées. Au lieu de dire à l'ordinateur exactement quelles caractéristiques définissent un chat, le système analyse de nombreux exemples étiquetés (des images marquées comme « chat » ou « pas chat ») et apprend progressivement quelles caractéristiques sont utiles pour les distinguer.

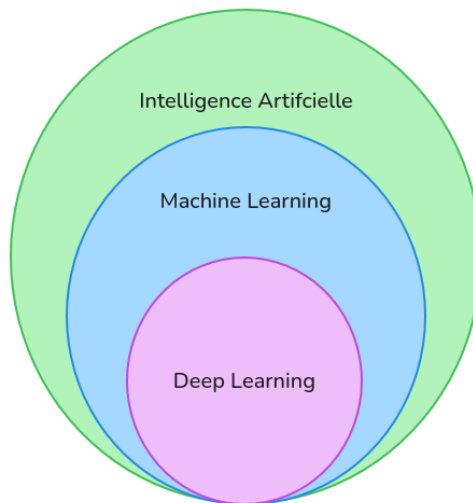


Figure 1: Relation entre IA, machine learning et deep learning

L'apprentissage profond est une sous-famille du machine learning. Ces deux approches partagent ce principe général d'apprentissage par l'exemple, mais elles diffèrent dans la façon dont les caractéristiques utiles sont identifiées. En machine learning classique, c'est l'humain qui effectue ce qu'on appelle le *feature engineering*, il choisit manuellement quelles caractéristiques la machine doit analyser (la longueur des oreilles,

le ratio du visage, etc.). En deep learning en revanche, le réseau de neurones profond découvre automatiquement quelles caractéristiques sont pertinentes, sans intervention humaine. C'est précisément l'utilisation de ces réseaux de neurones à plusieurs couches, capables d'apprendre des représentations de plus en plus abstraites, qui caractérise le deep learning. [1].

2.2 Concept Central d'Apprentissage Profond

2.2.1 Réseaux de Neurones

Un réseau de neurones est un modèle computationnel vaguement inspiré de l'organisation des neurones dans le cerveau humain. Tout comme le cerveau humain possède différentes régions associées à des tâches spécifiques, les réseaux de neurones sont organisés en couches ayant chacune un rôle particulier.

Il est composé de nombreuses unités de traitement simples appelées **neurones**, qui sont connectées entre elles. Ces connexions sont associées à des valeurs numériques appelées **poids**. Les données sont traitées à travers ces poids, qui déterminent l'importance de chaque entrée dans la prédiction finale. Le résultat est ensuite transmis à d'autres neurones du réseau.

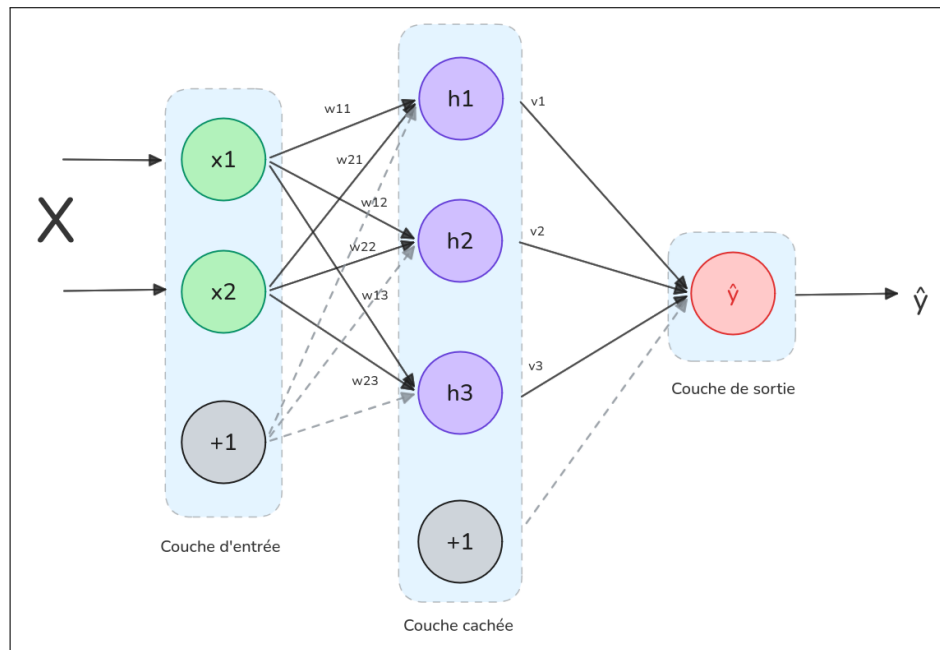


Figure 2: Schéma d'un réseau de neurones : propagation avant.

Le réseau reçoit deux variables d'entrée (x_1, x_2) et propage l'information vers trois neurones cachés (h_1, h_2, h_3) via des poids appris (w_{ij}), puis produit une prédiction $\hat{y}$.

Un réseau de neurones est organisé en différentes couches. La première couche, appelée couche d'entrée, reçoit les données brutes (*les features*). Dans le cas d'une image, cela peut correspondre aux valeurs numériques de ses pixels. L'information passe ensuite par une ou plusieurs couches cachées, où des représentations intermédiaires des données sont calculées. Enfin, la couche de sortie produit la prédiction du modèle, par exemple la probabilité de ce que contient l'image.

Dans un problème de classification, la couche de sortie produit généralement une ou plusieurs probabilités associées aux différentes classes possibles. Par exemple, dans un modèle qui doit reconnaître des animaux, la sortie peut contenir plusieurs valeurs représentant la probabilité que l'image corresponde à chaque catégorie

(chat, chien, etc.). La classe prédite par le modèle correspond alors à celle ayant la probabilité la plus élevée.

Ce qui rend le deep learning « profond » est le nombre de couches situées entre l'entrée et la sortie. Pour des problèmes complexes comme le traitement d'images, les réseaux de neurones nécessitent une structure de couches plus complexe avec différents types de couches cachées.

Pour produire des prédictions correctes, les poids et d'autres paramètres doivent être ajustés pendant la phase d'entraînement. La phase d'entraînement correspond au moment où le modèle apprend à partir d'exemples de données et modifie progressivement ses paramètres afin d'améliorer ses performances.

2.2.2 Fonction d'activation

L'exemple précédent du calcul entre les neurones fonctionne bien lorsque la frontière de décision ¹(*decision boundary*) est une ligne.

Cependant, avec des données plus complexes, une simple ligne ne suffit plus pour séparer correctement les différentes catégories. Il faut donc un modèle capable de créer des séparations plus complexes. Pour cela, on utilise des fonctions d'activation.

Les fonctions d'activation sont des expressions mathématiques appliquées à la sortie d'un neurone. Elles introduisent une non-linéarité dans le modèle, ce qui permet au réseau de neurones de représenter et d'apprendre des relations plus complexes dans les données.

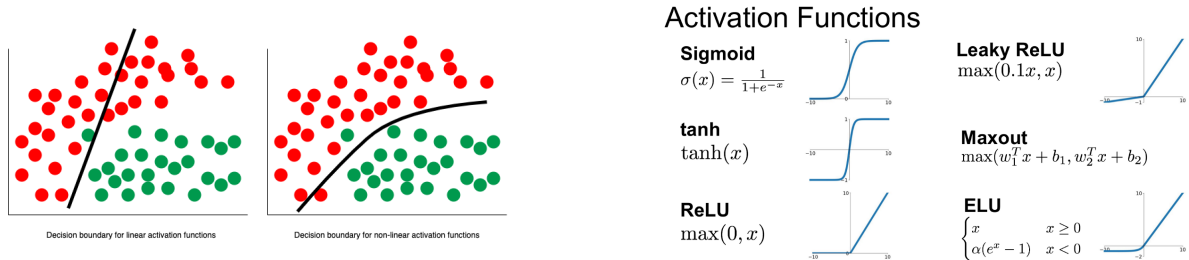


Figure 3: Fonctions d'activation et frontière de décision

2.2.3 loss function

La fonction 'Loss' représente la différence entre les valeurs réelles et les valeurs prédites par le modèle. Elle permet de mesurer la performance du modèle surtout dans les méthodes de **classification**.

Généralement il existe un graphe qui représente la valeur de 'Loss' à chaque itération (epoch) que le modèle effectue pendant l'apprentissage des données. ce graphe est appelée 'Courbe d'apprentissage', plus que la courbe descent (la loss diminue) , plus le modèle apprend et améliore ses prédictions.

¹La frontière de décision désigne la limite utilisée par le modèle pour séparer différentes catégories de données. Par exemple, dans un problème simple où l'on veut séparer deux groupes de points, cette frontière peut être une droite qui divise les deux classes.

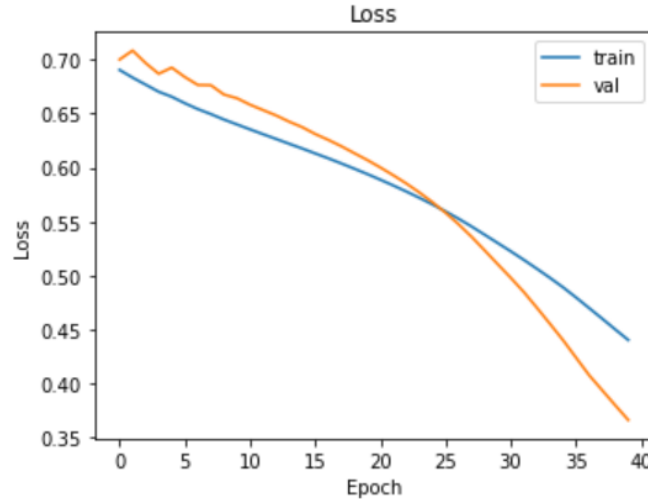


Figure 4: Exemple d'une courbe de Loss

Une méthode d'évaluation de loss utilisée dans les modèles est **Binary Cross-Entropy**, appelée aussi Log Loss. C'est une méthode utilisée surtout pour évaluer les problèmes de classification binaire (par exemple : la photo représente "chat" ou "pas chat").

Le modèle prédit une probabilité comprise entre 0 et 1, puis la fonction de loss pénalise les prédictions incorrectes.

Par exemple, si la probabilité prédite est 0.9 alors que la vraie valeur est 0, la pénalisation est élevée car la prédiction est très éloignée de la réalité. En revanche, si la probabilité est 0.1 et que la vraie valeur est 0, la pénalisation est faible, car la prédiction est proche de la valeur correcte.

Une autre métrique couramment utilisée est **l'accuracy** (précision), qui mesure simplement la proportion de prédictions correctes parmi l'ensemble des exemples. Contrairement à la loss, elle est directement interprétable : une accuracy de 90% signifie que le modèle prédit correctement 9 images sur 10.

2.2.4 descente de gradient

C'est un algorithme de minimisation d'erreur qui utilise les fonctions de loss.

l'idée générale : prenons une fonction linéaire simple :

$$y = ax + b$$

où y est **l'output** et x est **l'input**, a représente **le poids**, b représente **biais**.

L'algorithme met à jour les valeurs du biais et du poids jusqu'à obtenir un minimum de taux d'erreur afin de faire une prédiction la plus correcte de la droite ' y '.

La descente de gradient est utilisée dans les réseaux de neurones pour modifier itérativement le poids et le biais jusqu'à ce que le taux de loss soit significativement minimisé.

2.2.5 Backward Propagation

Jusqu'à présent, nous avons décrit le passage des données à travers le réseau afin de produire une prédiction, un processus appelé **forward propagation**. Une fois cette prédiction obtenue, il est nécessaire d'évaluer l'erreur via la fonction de perte (*loss function*) et d'ajuster les paramètres du modèle.

Backward Propagation est un algorithme qui optimise le modèle pendant l'entraînement qui est compris de plusieurs itérations appelée *epochs*. Comme les paramètres de calcul (comme les poids) sont initialisés aléatoirement, l'algorithme remonte le réseau au sens inverse de *forward propagation*, calcule les gradients.

Cherchons d'où vient l'erreur.

À chaque étape, on mesure à quel point chaque neurone a contribué à l'erreur, qui sera calculée grâce à la descente de gradient. Ensuite, on ajuste légèrement les paramètres de calcul pour réduire cette erreur.

Ce processus est répété plusieurs fois, ce qui permet au modèle de s'améliorer progressivement, en diminuant la fonction de perte.

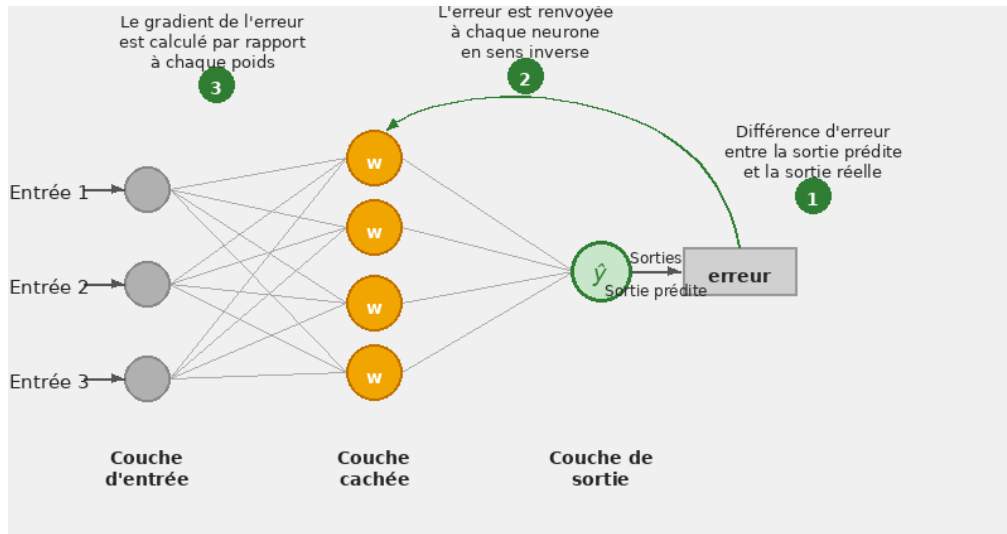


Figure 5: Réseaux de Neurones avec Backward propagation

2.2.6 Les Réseaux de Neurones Convolutifs

Les Réseaux de Neurones Convolutifs (CNN) sont largement utilisés pour les tâches de vision par ordinateur comme la classification d'images ou la détection d'objets. Ils appliquent des filtres de convolution sur l'image afin d'extraire automatiquement des caractéristiques visuelles. Les premières couches détectent des motifs simples (bords, textures), tandis que les couches plus profondes permettent d'identifier des structures plus complexes. [2].

3 Bibliographie

[lien rapport plmlatex](#)

References

- [1] Abid Ali Awan. Deep learning tutorial, 2023.
- [2] Shreya Rao. Implementing convolutional neural networks in tensorflow, 2024.